

ASSIGNMENT 3

Software Development and Integration

An analytical report with a practical case study of ReviewGuard

Project repository: <https://github.com/Svoppy/review-api>

Prepared October 2026

Scope note: The repository URL is the configured Git remote. GitHub was unreachable during preparation, so current public visibility and hosted workflow status are not verified.

1. Software development process and organization

Software development is a managed process that turns a stakeholder need into a dependable product and then keeps that product useful. A practical life cycle includes requirements discovery, architecture and design, implementation, verification, release, operation, and maintenance. These are related activities rather than isolated gates: test planning can reveal unclear requirements, and operational feedback can reopen design decisions [1].

Requirements should be expressed as observable behavior and quality constraints. For ReviewGuard, functional requirements include normalizing supported review datasets into a common schema, training models for sentiment and authenticity, exporting a model, and providing predictions through an API. Nonfunctional requirements include traceable inputs, repeatable splits, understandable errors, manageable inference cost, and explicit limits on scientific claims. Acceptance criteria turn these needs into checks: for example, a normalized record has a known schema, and an API response contains both task outputs or a transparent unavailable-model error.

Architecture divides the product into data, modeling, training, analysis, and API components. Interfaces between components reduce accidental coupling: normalized records are the boundary between source adapters and modeling; an exported checkpoint is the boundary between training and serving. Design decisions should be recorded when they affect reproducibility, data handling, or deployment. Implementation then proceeds in small changes that can be reviewed independently.

Verification combines unit tests for individual transformations, integration tests for component boundaries, and end-to-end checks for the user-visible workflow. Release means identifying the source revision and environment that produced an artifact. Maintenance includes dependency updates, bug fixes, monitoring, and retirement of unsupported paths. This lifecycle is iterative: a failed evaluation may lead to revised requirements, a split audit may lead to a data-processing change, and an API defect may lead to an interface revision.

2. Organizing iterative work and managing requirements

A small research project does not need a large hierarchy, but it still benefits from explicit ownership. The contributor who changes a module should state the problem, make a bounded change, add evidence, and document limitations. Issues can capture a defect or research question; a branch provides an isolated line of work; a pull request provides a review boundary; and a release or tagged revision identifies a shareable state. For a solo developer, these practices create a useful self-review trail.

An iterative workflow is appropriate because scientific requirements often become clearer as data and models are examined. A short cycle can be: select one requirement, inspect the relevant data and code, implement the smallest change, run focused tests, run the broader suite, review the diff, and update documentation. The cycle should include research review as well as coding. A high score is not meaningful if labels are inconsistent or data leakage invalidates the comparison.

The project should distinguish product requirements from research hypotheses. “The service returns two labels” is a product behavior. “Joint training improves both tasks” is a hypothesis requiring a controlled experiment and evidence. Code support does not establish a scientific result. ReviewGuard documentation makes this boundary important: the repository has broader model and dataset capabilities than the completed empirical evidence supports. The current claim should stay bounded to the actually prepared and reported study.

Change management also includes data and environment changes. A dataset update can alter class balance, duplicates, and evaluation leakage; it should be treated as a versioned input with provenance, not an invisible file replacement. A dependency update can alter numerical results or runtime behavior; it should be reviewed and tested. A useful completion record links the code revision, configuration, data identity, random seed, command, outputs, and test result. This makes the result inspectable by the original author and collaborators.

3. Choosing technologies and tools

Technology selection should begin with constraints, not popularity. For this project the criteria are: fit to multilingual review analysis; support for both machine learning and a web interface; maturity and documentation; ease of testing; portability across developer machines and CI; reproducibility; computational cost; licensing; and the ability to explain the choices to a research audience. A technology is suitable when it meets the actual requirements at an acceptable maintenance cost.

Python is selected because it connects data processing, machine learning, scientific analysis, and service development in one language. pandas supports tabular preparation; scikit-learn provides familiar baselines, splitting, and metrics; PyTorch supports model training; Hugging Face Transformers provides pretrained multilingual encoders and tokenizer/model interfaces; FastAPI and Pydantic provide a typed HTTP boundary; pytest runs the automated suite. This shared language reduces glue code and makes analysis steps easier to inspect.

The model choice is XLM-RoBERTa as a multilingual encoder for Russian and English text. It is a practical baseline, not a guarantee of domain validity: performance still depends on label quality, source composition, and evaluation design. The current architecture has separate sentiment and authenticity outputs, which directly represent the project's two prediction tasks. A classical baseline and single-task comparisons help test whether model complexity and joint learning add value.

The stack has costs. PyTorch and Transformers are large dependencies, model training may require accelerator hardware, and pretrained model weights can have separate licensing and access terms. An API is not a substitute for calibration or external validity. Those costs are managed by keeping data preparation and tests runnable without downloading a model, documenting the environment, using small deterministic fixtures, and describing the actual experimental scope. The chosen tools are therefore justified by task fit and research workflow, with limitations made visible.

4. Version control systems: Git and alternatives

Version control records file changes over time and supports collaboration. Centralized systems such as Subversion (SVN) keep the main history on a server. Distributed systems such as Git and Mercurial give each clone a local repository and history. A centralized server can provide a simple authority and access model; a distributed system allows local commits and branching without continuous network access, but teams must coordinate integration and resolve competing changes.

Git stores snapshots efficiently, supports branches and merges, and lets users inspect history, compare revisions, and recover prior states. A commit should represent a coherent change with a useful message. Branches allow work to proceed in parallel; merges or rebases integrate changes. Git's flexibility also creates risks: force-pushing shared history, committing generated data or secrets, and combining unrelated changes make collaboration harder. Ignore rules, code review, and clear commits reduce these risks [2].

Git and GitHub are not the same thing. Git is a version control system that runs locally or with any compatible remote. GitHub is a hosted collaboration platform built around Git repositories. It adds repository permissions, pull requests, issue tracking, project discussions, and hosted automation. Git can be used without GitHub; GitHub cannot replace the local history and branching model of Git. Other hosting platforms provide similar services, and a Git repository can have multiple remotes.

ReviewGuard already has a Git repository and a configured remote at <https://github.com/Svoppy/review-api>. The active local branch is `dissertation-mvp`. A configured remote proves the intended destination, but does not prove public visibility or that the latest local files have been pushed. Since this environment could not resolve GitHub, live visibility and the current remote issue list could not be confirmed during preparation. The report includes the shareable repository and workflow URLs, with that verification boundary stated explicitly.

5. GitHub workflow, documentation, and project governance

A simple GitHub collaboration flow begins with an issue describing a problem or change. A contributor creates a branch, commits a focused implementation, and opens a pull request. Reviewers discuss the diff and CI results; after changes are accepted, the branch is merged. This links the reason for a change to its implementation and verification. GitHub issues can also track research tasks, provided that sensitive review text, personal data, and credentials are never posted there [3, 4].

A public repository should explain its purpose, installation, commands, expected inputs and outputs, limitations, contribution path, and license. A license is legally important: without an explicit license, public visibility alone does not grant general permission to reuse the code. This project includes an MIT license for repository code and documentation; model and dataset licenses remain separate and must be checked at their original sources. The bug report template asks for a minimal reproduction and environment while warning users not to expose private review data.

The repository README links to the assignment report, license, issue tracker, and CI workflow. The project URL is <https://github.com/Svoppy/review-api> and the workflow configuration is <https://github.com/Svoppy/review-api/blob/dissertation-mvp/.github/workflows/ci.yml>. The practical project uses Git for history, GitHub as the intended remote and collaboration surface, and Actions for hosted checks. A workflow file in the repository is configuration; it is not proof that a hosted run succeeded. The first successful run and public visibility should be checked on GitHub after the changes are pushed. The current network restriction prevents claiming that step as complete.

For research software, repository governance also includes stewardship of data. Large datasets, model checkpoints, credentials, and private inputs should not be committed by default. ReviewGuard's ignore rules exclude generated model artifacts and raw/processed data. Public code should point to lawful data sources and document preparation steps without redistributing material that the project does not have rights to share. These practices help keep the repository useful and reduce accidental disclosure.

6. Continuous integration and continuous delivery

Continuous integration (CI) automatically builds and tests proposed changes so integration defects are detected close to the change. A typical pipeline checks out the revision, installs a defined runtime, installs the project, runs static checks and tests, and reports a result. CI is valuable when tests are deterministic, fast enough to run routinely, and designed to catch meaningful failures. It cannot prove scientific validity, data representativeness, security, or correct deployment by itself.

Continuous delivery (CD) keeps software in a deployable state and can publish a candidate artifact after required checks. Continuous deployment goes further by automatically releasing every change that passes the pipeline. These terms describe different release policies. ReviewGuard's current workflow is a CI pipeline: it tests changes but does not deploy a public service or publish a package. A safe next step for delivery would be to build a versioned artifact after a reviewed release tag, then require an explicit environment and credentials for deployment.

The added `.github/workflows/ci.yml` runs on pushes, pull requests, and manual dispatch. It uses Ubuntu and Python 3.12, installs the editable package and pytest, then runs the repository test suite with `PYTHONPATH=src`. The workflow has read-only repository contents permission. Python 3.12 is selected because `.python-version` pins 3.12.7 as the reproducibility target; the repository separately records a locally verified Python 3.14.2 environment. The pinned scientific stack may make installation slower, but CI caching and deterministic tests keep routine feedback practical.

A pipeline should evolve with the project. Useful additions include Ruff linting, a minimum-supported Python version matrix, dependency vulnerability checks, packaging validation, and a separately controlled release job. Each addition should protect a stated quality requirement. For machine learning, CI should use small local fixtures and avoid downloading large models or datasets; model performance experiments belong in a reproducible experiment process rather than an ordinary unit-test job. The Actions setup, workflow syntax, and run history are documented by GitHub [5, 6].

7. Scientific software requirements and reproducibility

Scientific software has two obligations: it must behave as software and support credible research. Requirements therefore include data provenance, clear label definitions, transparent exclusions, explicit preprocessing, stable evaluation splits, and a record of parameters and software versions. The data path should be inspectable from raw source through normalization, splitting, training, evaluation, and reporting. A researcher should be able to identify which code and input produced a result.

Environments are part of the method. A repository should state the supported Python version and dependency requirements. Exact lockfiles or container images can strengthen repeatability when needed, while flexible dependency ranges ease installation but may allow drift. Random seeds, deterministic split files, configuration files, and manifests record choices. Where hardware or framework behavior prevents bit-for-bit reproduction, the documentation should describe the expected tolerance and limitation rather than promise exact identity.

Libraries should be selected for a clear scientific role and tracked with versions. Data parsing libraries, numerical libraries, model frameworks, metrics, and web frameworks may all affect outputs. Tests should include known-answer examples, boundary cases, schema validation, and leakage checks. Integration tests should cover file-to-model and model-to-API boundaries. A test suite can establish that code follows a defined contract; it cannot establish that a dataset's labels are correct or that the study answers its research question.

Scientific best practices include version-controlling scripts, automating repeated steps, writing readable code, checking intermediate results, sharing code and outputs where legally possible, and documenting how to reproduce figures and tables. Sandve and colleagues emphasize version control and public access to scripts, runs, and results; Wilson and colleagues emphasize testing, automation, documentation, and collaboration [7, 8]. ReviewGuard applies these principles through data audits, experiment configurations, scripted reports, tests, and scope statements.

8. ReviewGuard: practical scientific software module

ReviewGuard is a research-oriented system for joint analysis of review sentiment and review authenticity. It includes dataset adapters and normalization into a common record schema; split and audit utilities; classical, single-task, and multitask model training; metric and robustness analysis; checkpoint export; a FastAPI `/analyze` endpoint; and a lightweight browser interface. These existing components make the repository a substantial practical example rather than a disconnected classroom toy.

The implementation stack targets Python 3.12.7 as pinned in `.python-version` (the repository documents local verification under Python 3.14.2); pandas and dataset adapters for preparation; scikit-learn for baselines and evaluation; PyTorch and Transformers for multilingual neural models; FastAPI/Pydantic for serving; pytest for automated checks; Git and GitHub for version history and collaboration; and GitHub Actions for CI. The stack is selected because it covers the research workflow from data to service while keeping the analysis in a single language. Heavy model training is not required for the CI test suite.

The repository contains tests for data pipelines, datasets, inference artifacts, training metrics and runtime, statistical analyses, robustness, and report generation. The new CI workflow runs those tests for proposed changes. The README now connects the scientific tool to the assignment and points to the report, CI, license, and issue tracker. The MIT license supports reuse of the code subject to its terms; original dataset and pretrained model licenses remain applicable.

The implementation must be interpreted within its scientific boundary. The code supports more sources and experiments than the locally reported evidence currently covers. The README and research documentation state that the completed evidence is a bounded low-resource study and does not justify broad superiority or robustness claims. This distinction is a quality feature: the software can facilitate future evaluation without claiming that future work is already complete.

9. Verification, challenges, and conclusion

Local verification for this assignment consists of inspecting the complete one-page brief, reviewing the repository structure and existing tests, adding the workflow and project materials, and running the test suite. The expected CI command is ``PYTHONPATH=src python -m pytest -q``; a successful local run provides evidence about this checkout, while a green GitHub Actions run must be confirmed on the remote after publication. The GitHub host could not be reached from this environment, so push status, public visibility, issue tracker contents, and hosted workflow results remain unverified.

A challenge is the scale of the existing research codebase and its dependency stack: model libraries increase installation time, and meaningful model training is too expensive for every commit. The solution is to keep CI focused on deterministic tests and treat large experiments as separately configured research runs. Another challenge is the difference between software capability and empirical evidence. Documentation must not imply that model support, a dataset adapter, or a passing unit test establishes a scientific conclusion.

The work demonstrates a complete development approach: requirements and architecture guide implementation; technology selection follows research needs; Git records changes; GitHub provides the intended public collaboration location; tests check contracts; and Actions automates continuous integration. The repository already provides the core scientific application, and the added CI, license, issue template, report, and README links make that work easier to inspect and reuse.

The next completion step is operational: push the prepared changes to the user's GitHub repository, verify that the repository is public, inspect the issue page and license detection, and confirm a green Actions run. These remote checks could not be performed offline. Subject to that verification, the project meets the assignment's practical and theoretical goals while remaining honest about its research limits.

References

- [1] SWEBOK, Chapter 8: Software Engineering Process. [Open reference](#).
- [2] Chacon and Straub, *Pro Git*, 2nd ed. [Open book](#).
- [3] GitHub Docs, GitHub flow. [Open docs](#).
- [4] GitHub Docs, repositories and issues. [Repositories; issues](#).
- [5] GitHub Docs, continuous integration. [Open docs](#).
- [6] GitHub Docs, workflows. [Open docs](#).
- [7] Sandve GK, et al. Ten Simple Rules for Reproducible Computational Research. PLoS Comput Biol. 2013;9(10):e1003285. [doi:10.1371/journal.pcbi.1003285](#).
- [8] Wilson G, et al. Best Practices for Scientific Computing. PLoS Biol. 2014;12(1):e1001745. [doi:10.1371/journal.pbio.1001745](#).
- [9] GitHub Docs, licensing a repository. [Open docs](#).